

Spring Boot REST & JWT — Complete Hands-on Task Checklist

Consolidated from: 1_spring-rest-handson, 2_spring-rest-handson, 3_spring-rest-handson, 4_spring-rest-handson, 5_JWT-handson. Every task box below is a discrete, gradeable step — tick it off, commit, and push to your GitHub repo (COGNIZANT) as you complete it.

Document 1 — Spring Core Basics (spring-rest-handson #1)

Hands-on 1: Create a Spring Web Project using Maven

- Go to <https://start.spring.io/>
- Set Group = **com.cognizant**, Artifact Id = **spring-learn**
- Select Spring Boot DevTools and Spring Web dependencies
- Generate and download the project as a ZIP
- Extract the ZIP into your Eclipse workspace root folder
- Build the project via `mvn clean package` (with proxy flags if required)
- Import into Eclipse: File > Import > Maven > Existing Maven Projects
- Add a log statement inside `main()` of `SpringLearnApplication` to verify startup
- Run `SpringLearnApplication` and confirm it starts cleanly
- Review with SME: `src/main/java`, `src/main/resources`, `src/test/java`, `@SpringBootApplication`, `pom.xml` dependency tree

Hands-on 2: Spring Core — Load SimpleDateFormat from Spring Configuration XML

- Create **date-format.xml** in `src/main/resources` of `spring-learn`
- Define a `<bean id="dateFormat" class="java.text.SimpleDateFormat">` with `constructor-arg value="dd/MM/yyyy"`
- Add method **displayDate()** in `SpringLearnApplication.java`
- Inside it, build `ApplicationContext` via `ClassPathXmlApplicationContext("date-format.xml")`
- Retrieve the bean using `context.getBean("dateFormat", SimpleDateFormat.class)`
- Parse the string '31/12/2018' and print the parsed `Date` to console
- Run as a Java Application and verify console output
- (Troubleshooting) If Tomcat port conflicts, set an alternate `server.port` in `application.properties`

Hands-on 3: Spring Core — Incorporate Logging

- Create `application.properties` in `src/main/resources` (if not present)
- Add: `logging.level.org.springframework=info`
- Add: `logging.level.com.cognizant.springlearn=debug`
- Add the custom `logging.pattern.console` format line
- Import `org.slf4j.Logger` and `org.slf4j.LoggerFactory` in `SpringLearnApplication.java`
- Declare a static final `Logger` `LOGGER` field
- Wrap every method with an `INFO "START"` log at entry and `"END"` log at exit; use `DEBUG` for data values
- Replace any `System.out.println()` usage with logger calls (never use `println` going forward)

Hands-on 4: Spring Core — Load Country from Spring Configuration XML

- Create **country.xml** Spring config for one country (code + name)
- Define a `<bean id="country" class="com.cognizant.springlearn.Country">` with property tags for code/name
- Create `Country.java`: instance vars `code` & `name`
- Add a no-arg constructor with a `DEBUG` log "Inside Country Constructor."
- Generate getters/setters, each logging at `DEBUG` level

- Generate toString()
- Add displayCountry() method in SpringLearnApplication.java using ClassPathXmlApplicationContext + getBean("country", Country.class)
- Log the country details at DEBUG
- Call displayCountry() from main()
- Run and verify constructor/getter/setter logs in console

Hands-on 5: Spring Core — Singleton vs Prototype Scope

- In displayCountry(), fetch the country bean a second time from the same context (anotherCountry)
- Confirm the constructor is invoked only ONCE (singleton default) via logs
- Add scope="prototype" attribute to the <bean> tag in country.xml
- Re-run and confirm the constructor is now invoked TWICE (prototype)

Hands-on 6: Spring Core — Load list of countries from Spring Configuration XML

- Add 4 separate country <bean> definitions (US, DE, IN, JP) in country.xml
- Define an ArrayList bean "countryList" using <constructor-arg><list><ref bean=...>
- Add method displayCountries() in SpringLearnApplication.java
- Read the countryList bean and log each country at DEBUG level
- Include START/END INFO logs in the new method

Document 2 — REST Fundamentals (spring-rest-handson #2)

Warm-up: HTTP Request/Response Inspection (no code)

- Read RFC 7230 sample request/response (page 7) and identify method, resource, HTTP version
- Open Chrome DevTools (F12) > Network tab, load a page, and inspect General / Request / Response headers

Hands-on: Hello World RESTful Web Service

- Create controller **com.cognizant.spring-learn.controller.HelloController**
- Add `@RestController` and a `@GetMapping("/hello")` method: `public String sayHello()`
- Return the hard-coded string "Hello World!!"
- Include START and END logs inside `sayHello()`
- Test via browser at `http://localhost:8083/hello`
- Test via Postman and inspect the Headers tab

REST — Country Web Service (single country)

- Create **CountryController** with `@RequestMapping` and method `getCountryIndia()`
- URL: `/country` — load the India bean from Spring XML config and return it
- Verify JSON response: `{"code":"IN","name":"India"}`
- Inspect response headers in DevTools Network tab and in Postman

REST — Get all countries

- In **CountryController** add `@GetMapping("/countries")` method `getAllCountries()`
- Load the full country list from `country.xml` and return it
- Verify JSON array response matches all 4 countries

REST — Get country based on country code

- Add `@GetMapping("/countries/{code}")` method `getCountry(String code)` using `@PathVariable`
- Create `CountryService.getCountry(String code)` that does a case-insensitive match against the country list
- (Optional) implement the match with a lambda instead of manual iteration
- Test `http://localhost:8083/country/in` and confirm correct India JSON is returned

REST — Get country exceptional scenario

- Create exception class **CountryNotFoundException** in `service.exception` package
- Annotate it with `@ResponseStatus(value = HttpStatus.NOT_FOUND, reason = "Country not found")`
- Throw `CountryNotFoundException` from `CountryService.getCountry()` when code is not found
- Add "throws" clause to `CountryController.getCountry()`
- Test with Postman and with: `curl -i http://localhost:8090/country/az`
- Confirm 404 JSON error body matches the expected sample

MockMVC — Test get country service

- Autowire **CountryController** in `SpringLearnApplicationTests` and assert it's not null in `contextLoads()`

- Run via right-click > Run As > JUnit Test, then also via: mvn clean test
- Add `@AutoConfigureMockMvc` and autowire `MockMvc`
- Write `testGetCountry()`: perform `GET("/country")`, assert `status().isOk()`
- Assert `jsonPath("$.code").exists()` and `jsonPath("$.code").value("IN")`
- Add equivalent assertions for the "name" field = "India"

MockMVC — Test get country service for exceptional scenario

- Add `testGetCountryException()` method
- Assert `status().isBadRequest()` / correct status and `status().reason(...)` for the not-found case

Document 3 — Employee & Department REST Services (spring-rest-handson #3)

No quiz for this session. This doc is mostly a design brief — the concrete steps are the ones below.

Create static employee list data (Spring XML configuration)

- In **employee.xml**, create 1–2 additional Department beans
- Create 4 more Employee instances (reuse sample data from the Angular app)
- Reuse existing Skill beans instead of duplicating them
- Wrap all employee instances into an ArrayList bean
- In EmployeeDao: add static field EMPLOYEE_LIST of type ArrayList<Employee>
- Add a constructor that reads the employee list from the XML config and sets EMPLOYEE_LIST
- Add method getAllEmployees() returning EMPLOYEE_LIST

Create REST service to get all employees

- Change EmployeeService annotation from @Component to @Service
- Add getAllEmployees() that calls employeeDao.getAllEmployees() and returns the list
- Annotate the method with @Transactional
- In EmployeeController add @GetMapping("/employees") method getAllEmployees()
- Have it call employeeService.getAllEmployees() and return the result
- Test the /employees endpoint in Postman

Create REST service for department

- Create DepartmentController with @GetMapping("/departments") method getAllDepartments()
- Create DepartmentService.getAllDepartments()
- Create DepartmentDao with static DEPARTMENT_LIST populated from Spring XML config, plus getAllDepartments()
- Test the /departments endpoint in Postman
- Verify the service call by checking the application logs

Document 4 — POST / PUT / DELETE & Validation (spring-rest-handson #4)

Apply REST naming conventions to CountryController

- Move the shared URL to class level: `@RequestMapping("/countries")`
- Ensure GET all, GET by id, POST, PUT, DELETE all follow the resource-plural convention
- Refactor CountryController to match this standard (apply going forward to all new services)

POST — handle Country creation (step 1: bare endpoint)

- Add method `addCountry()` with `@PostMapping()` and void return type
- Log "Start" inside the method
- Start the app and invoke: `curl -i -X POST -s http://localhost:8090/countries`
- Confirm "Start" appears in the console and response is HTTP/1.1 200 with Content-Length: 0
- Repeat the test using Postman

POST — read Country payload as a bean

- Change signature to: `public Country addCountry(@RequestBody Country country)`
- Log the received country details
- Return the country object so the response can be verified
- Test with `curl -i -H 'Content-Type: application/json' -X POST -s -d '{...}' http://localhost:8090/countries`
- Try a payload with a misspelled field (e.g. "nae" instead of "name") and observe null is bound silently

Validate country code (manual javax.validation approach)

- Add `@NotNull` and `@Size(min=2,max=2)` annotations on `Country.code`
- In `addCountry()`, build a `ValidatorFactory/Validator` and call `validator.validate(country)`
- Collect violation messages into a `List<String>`
- If violations exist, throw `ResponseStatusException(HttpStatus.BAD_REQUEST, errors.toString())`
- Re-test with an invalid (1-character) country code and confirm HTTP 400 with the expected error JSON

Global exception handler for validation errors

- Create `GlobalExceptionHandler` extends `ResponseEntityExceptionHandler`, annotated `@ControllerAdvice`
- Override `handleMethodArgumentNotValid(...)` with a START log
- Add `@Valid` to `addCountry(@RequestBody @Valid Country country)`
- Remove the manual `ValidatorFactory` code added in the previous task
- Re-test with an invalid country code and confirm the global handler (not the controller) logs "Start"
- Build the response body map (timestamp, status, errors list) inside `handleMethodArgumentNotValid`
- Confirm the final JSON error shape matches the expected sample `{timestamp,status,errors:[...]}`

Implement REST service for updating an Employee

- Add validation annotations to Employee (id, name, salary, permanent, dateOfBirth incl. `@JsonFormat` pattern)

- Add validation annotations to Department (id, name)
- Add validation annotations to Skill (id, name)
- Create EmployeeNotFoundException with an @ResponseStatus annotation
- Add EmployeeDao.updateEmployee() — modify the list or throw EmployeeNotFoundException
- Add EmployeeService.updateEmployee() calling the DAO method
- Add EmployeeController.updateEmployee(@RequestBody @Valid Employee employee) with @PutMapping, following URL conventions
- Add handleHttpRequestMessageNotReadable() to GlobalExceptionHandler for bad numeric-field input (e.g. non-numeric id)
- Test update via Postman with full payload including department + skills
- Re-call GET /employees to confirm the update persisted
- Add a MockMvc test for the exceptional (not-found) update scenario

Implement REST DELETE service for Employee

- Add EmployeeController delete endpoint (proper @DeleteMapping URL convention)
- Add EmployeeService delete method
- Add EmployeeDao delete logic — remove from list or throw EmployeeNotFoundException if id not found
- Test deletion via Postman and confirm the employee is removed from subsequent GET /employees calls

Document 5 — Spring Security & JWT (JWT-handson #5)

Secure all web services with Spring Security (HTTP Basic, auto-generated password)

- Add spring-boot-starter-security dependency to pom.xml
- Rebuild with mvn clean package (include proxy flags)
- Eclipse: right-click project > Maven > Update Project
- Create package com.cognizant.spring-learn.security
- Create SecurityConfig class extending WebSecurityConfigurerAdapter
- Annotate with @Configuration and @EnableWebSecurity
- Start the app; call curl -s http://localhost:8090/countries and confirm 401 Unauthorized
- Find the auto-generated password in the startup logs
- Re-call the service with curl -s -v -u user:<generated-password> ... and confirm 200 + country list

Create named users/roles (admin / user, password 'pwd')

- Override configure(AuthenticationManagerBuilder auth) with in-memory users: admin/ROLE_ADMIN and user/ROLE_USER
- Add a passwordEncoder() @Bean returning new BCryptPasswordEncoder()
- Override configure(HttpSecurity) to disable csrf, enable httpBasic, and restrict /countries to hasRole("USER")
- Test success: curl -s -u user:pwd http://localhost:8090/countries
- Test wrong password: curl -s -u user:pwd1 ... and confirm 401
- Test wrong role: curl -s -u admin:pwd ... and confirm 403 Forbidden
- Run curl -s -v -u admin:pwd ... and locate the Authorization: Basic header value
- Base64-decode that header value online and confirm it reveals "admin:pwd" (illustrates why Basic Auth alone isn't secure)

Understand JWT structure (exercise, no code)

- Open the JWT Wikipedia structure article and <https://jwt.io/> side by side
- Paste the sample header and payload from Wikipedia into jwt.io's Decoded panel
- Enter "secretkey" as the signature secret
- Confirm the Encoded token generated by jwt.io matches the Wikipedia sample

Create authentication controller returning a JWT (step 1: scaffold)

- Create AuthenticationController in the controller package
- Add authenticate() method mapped to @GetMapping("/authenticate")
- Read the Authorization header via @RequestHeader("Authorization") String authHeader
- Return type Map<String,String>; include START/END logs and a DEBUG log of authHeader
- Return a map with key "token" and an empty string value (placeholder)
- In SecurityConfig, add .antMatchers("/authenticate").hasAnyRole("USER","ADMIN") after the /countries rule
- Test: curl -s -u user:pwd http://localhost:8090/authenticate → expect {"token":""}
- Confirm the Authorization header (Basic + Base64 of user:pwd) appears in the logs

Decode username/password from the Authorization header

- Add private method `getUser(String authHeader)` in `AuthenticationController`
- Strip the "Basic " prefix and Base64-decode using `Base64.getDecoder().decode(...)`
- Convert the resulting `byte[]` to a `String` and extract the substring before the colon (the username)
- Return the username; add `DEBUG` logs throughout
- Call `getUser()` from `authenticate()` and re-test, confirming the decoded username appears in logs

Generate a JWT token for the authenticated user

- Add the `jjwt` Maven dependency (`io.jsonwebtoken:jjwt:0.9.0`)
- Rebuild with `mvn package` and update the Eclipse project; verify `jjwt` appears in the dependency tree
- Add private method `generateJwt(String user)` using `Jwts.builder()`
- Set subject (user), issued-at (now), and expiration (now + 20 minutes)
- Sign with `SignatureAlgorithm.HS256` and secret "secretkey"; `compact()` to produce the token string
- Call `generateJwt()` from `authenticate()` and place the token into the response map (replacing the empty string)
- Re-test `curl -s -u user:pwd http://localhost:8090/authenticate` and confirm a real JWT is returned

Authorize all requests based on the JWT (custom security filter)

- Create `JwtAuthorizationFilter` in `com.cognizant.springlearn.security`, extending `BasicAuthenticationFilter`
- Add constructor taking `AuthenticationManager`, calling `super(...)` with `START/DEBUG` logs
- Override `doFilterInternal()`: if Authorization header is missing or doesn't start with "Bearer ", just continue the chain
- Otherwise call `getAuthentication(req)` and set it on `SecurityContextHolder` before continuing the chain
- Implement private `getAuthentication()`: parse the Bearer token with `Jwts.parser().setSigningKey("secretkey").parseClaimsJws(...)`
- Extract the subject (username); return a `UsernamePasswordAuthenticationToken`, or null on `JwtException`
- In `SecurityConfig`'s second `configure(HttpSecurity)`: comment out the old `/countries` `antMatcher` rule
- Keep `.antMatchers("/authenticate").hasAnyRole("USER","ADMIN")`; add `.anyRequest().authenticated()`
- Register the filter: `.addFilter(new JwtAuthorizationFilter(authenticationManager()))`
- Get a fresh token via `curl -s -u user:pwd http://localhost:8090/authenticate`
- Call a protected endpoint with `curl -s -H "Authorization: Bearer <TOKEN>" http://localhost:8090/countries` and confirm success
- Tamper with the token and confirm the request is now rejected

Push Completed Work to Your GitHub Repo (COGNIZANT)

The screenshot you shared shows a repo already containing exercise1...exercise15+ folders. Use this generic workflow for each exercise/project once its hands-on tasks above are complete (fill in your actual local project path and exercise folder name).

- `cd` into your local project folder for that exercise (e.g. `spring-learn`)
- `git init` (only if this folder isn't already a git repo)
- `git remote add origin <your-COGNIZANT-repo-URL>` (skip if already added — check with: `git remote -v`)
- `git checkout -b exerciseN-your-exercise-name` (or reuse the matching existing folder/branch)
- `git add .`
- `git commit -m "Complete exerciseN: "`
- `git pull origin main --rebase` (sync first to avoid conflicts)
- `git push origin main` (or your feature branch, then open a PR)
- Refresh github.com to confirm the folder/files now appear under COGNIZANT

Tell me your local folder structure / how far you've gotten and I can give you the exact git commands (or write them as a ready-to-run script) instead of the generic version above.